

LA CANTINA
System Design Document
Versione 1.0



Progetto: LA CANTINA	Versione: 1.0
Documento: System Design Document	Data: 22/11/2025

Coordinatore del progetto:

Nome	Matricola
GRAZIOSO Fabrizio	0512122950

Partecipanti:

Nome	Matricola
LIGUORI Alessandro	0512119887
CICALESE Gabriele	0512116443
GRAZIOSO Fabrizio	0512122950

Scritto da:	GRAZIOSO Fabrizio
--------------------	-------------------

Revision History

Data	Versione	Descrizione	Autore
22/11/2025	1.0	Scrittura documento	Grazioso, Cicalese, Liguori

Sommario

1.	Introduzione	4
1.1.	Purpose of the system	4
1.2.	Design goal	4
1.3.	Definitions, acronyms and abbreviations	4
1.4.	References.....	4
1.5.	Overview.....	4
2.	Current software architecture.....	4
3.	Proposed software architecture	4
3.1.	Overview.....	4
3.2.	Subsystem decomposition.....	5
3.3.	Hardware/Software mapping	6
3.4.	Persistent data management.....	7
3.5.	Access Control and security.....	7
3.6.	Global Software control.....	7
3.7.	Boundary Conditions	8
4.	Subsystem services	8

1. Introduzione

1.1. *Purpose of the system*

Lo scopo del sistema LaCantina è fornire una piattaforma di e-commerce specializzata nella vendita di prodotti agroalimentari a km 0 provenienti dalla Campania. Il sistema permette la gestione del catalogo prodotti, carrello, ordini e funzionalità amministrative.

1.2. *Design goal*

- **Affidabilità:** gestione coerente degli ordini e dello stock
- **Modificabilità / Manutenibilità:** architettura a layer, DAO isolati
- **Efficienza:** risposta rapida delle operazioni di consultazioni prodotti
- **Scalabilità:** supporto concorrente a più utenti
- **Usabilità:** interfaccia semplice per utenti ed amministratori
- **Sicurezza:** prevenzione accessi non autorizzati e SQL injection

1.3. *Definitions, acronyms and abbreviations*

- **MVC:** Model–View–Controller
- **DAO:** Data Access Object
- **SDD:** System Design Document
- **RAD:** Requirements Analysis Document
- **UC:** Use Case
- **DBMS:** Database Management System

1.4. *References*

- RAD LaCantina
- Slides Ingegneria del Software M3 – System Design

1.5. *Overview*

Il documento fornisce una descrizione dell'architettura software adottata dal progetto LaCantina, fornendo informazioni sulla divisione dei vari sub-system, il mapping tra componenti hardware e software ed in generale un approfondimento sul sistema proposto

2. Current software architecture

L'attuale sistema di mercato, come emerso dall'indagine, è dominato da un modello di consumo poco sostenibile. Si basa sull'acquisto diffuso, anche a prezzi elevati, di prodotti alimentari di dubbia qualità i quali, spesso derivano da processi industriali intensi. In questo modello produttivo, emergono due principali problemi, il primo riguardando il danno ambientale derivato dall'insostenibilità di questo tipo di produzione, il secondo è la mancata valorizzazione dei prodotti locali.

3. Proposed software architecture

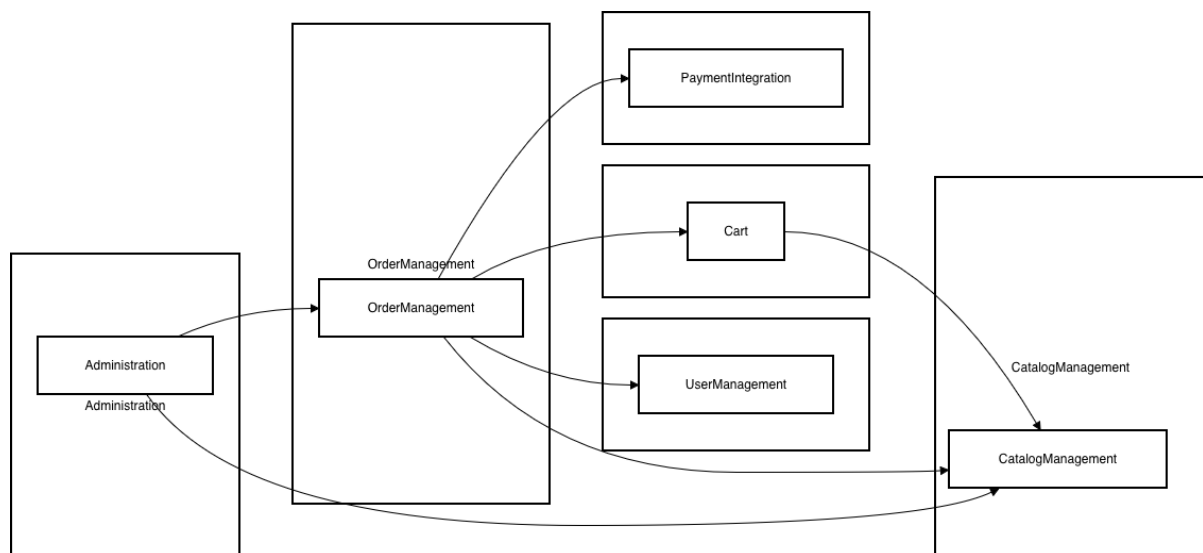
3.1. *Overview*

L'architettura software di "La Cantina" è strutturata secondo lo stile Model-View-Controller (MVC), organizzata su tre livelli logici distinti per garantire una chiara separazione delle responsabilità. Al

vertice troviamo il Livello di Presentazione (View), che gestisce l'interfaccia utente tramite tecnologie web standard (JSP, HTML, CSS, JavaScript). Questo livello si occupa di raccogliere gli input (come nei form di login o nei filtri di ricerca) e di presentare i risultati all'utente, visualizzando elementi come il catalogo prodotti, il carrello e le dashboard amministrative. Il coordinamento delle operazioni è affidato al Livello di Logica Applicativa (Control). Qui, le Servlet agiscono come controller implementando i casi d'uso (UC1–UC9): esse ricevono le richieste, validano i dati, gestiscono eventuali errori e invocano i servizi necessari, decidendo quale vista restituire all'utente. Infine, il Livello di Gestione Dati (Model) rappresenta lo stato del sistema. È costituito dalle classi Entità (Utente, Prodotto, Ordine) e dalle classi DAO, che astraggono la comunicazione con il database MySQL, gestendo le operazioni CRUD e l'aggiornamento delle informazioni. L'adozione del pattern MVC è motivata dalla sua perfetta aderenza alla natura web del progetto e allo stack tecnologico scelto, facilitando la manutenibilità grazie al disaccoppiamento tra presentazione, logica di business e persistenza dei dati.

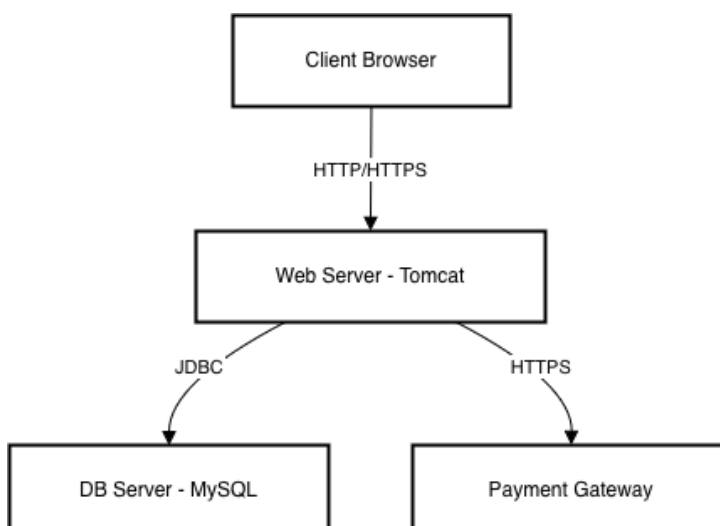
3.2. *Subsystem decomposition*

- **UserManagement:** Gestisce l'intero ciclo di vita dell'identità utente, incluse l'autenticazione, la registrazione e la gestione delle sessioni. Il modulo distingue i ruoli (utente/amministratore) ed è progettato per isolare completamente la logica delle credenziali e della sicurezza dal resto dell'applicazione.
- **CatalogManagement:** Centralizza tutte le operazioni di lettura relative ai prodotti. Si occupa di gestire e fornire informazioni su prezzi, provenienza, aziende fornitrici e stock logico, raccogliendo in un unico punto le funzionalità necessarie alla consultazione del catalogo.
- **Cart:** Gestisce il carrello della spesa memorizzato nella sessione utente (aggiunta, rimozione e modifica quantità). Essendo una struttura temporanea, viene separato dagli ordini persistenti per specializzarne il comportamento e mantenerlo legato alla durata della sessione.
- **OrderManagement:** Racchiude la logica di business relativa alla trasformazione del carrello in un ordine confermato. Si occupa della creazione dell'ordine, della registrazione delle righe, dell'aggiornamento dello stock fisico e della gestione dei cambi di stato dell'ordine.
- **Administration:** Raggruppa le funzionalità privilegiate riservate all'amministratore, quali la modifica di stock e prezzi e la gestione CRUD dei prodotti (UC5-UC9). Questo modulo serve a separare nettamente le operazioni critiche di backend da quelle accessibili all'utente standard.
- **PaymentIntegration:** Incapsula l'interazione con i gateway di pagamento esterni (o simulati). Fornendo un'interfaccia interna standardizzata per le autorizzazioni, questo sottosistema isola le dipendenze esterne, facilitando la futura integrazione o sostituzione dei provider di pagamento



3.3. *Hardware/Software mapping*

- **Nodo Client:** Rappresenta il punto di accesso per l'utente finale tramite dispositivi personali (PC, laptop, smartphone). Il software di riferimento è un browser web moderno (es. Chrome, Firefox), responsabile dell'invio delle richieste HTTP/HTTPS e della visualizzazione delle pagine HTML/CSS/JS renderizzate dal server.
- **Nodo Web Server:** Costituisce il cuore elaborativo del sistema, ospitato su un server che esegue Apache Tomcat e l'applicazione LaCantina.war. Questo nodo contiene tutti i package applicativi (Boundary, Controller, Entity, DAO) e gestisce centralmente la logica di business, le sessioni utente e l'orchestrazione delle comunicazioni verso il database e i servizi esterni
- **Nodo Database Server:** È dedicato alla persistenza dei dati e ospita il DBMS MySQL. Questo nodo si occupa di memorizzare e recuperare in modo efficiente tutte le informazioni strutturate del sistema, tra cui le anagrafiche utenti, il catalogo prodotti e lo storico degli ordini.
- **Nodo Payment Gateway:** Identifica un sistema esterno (reale o mock) interfacciato dall'applicazione. Il suo unico ruolo è ricevere le richieste di transazione finanziaria e restituire l'esito (successo o fallimento) al Web Server, senza risiedere nell'infrastruttura fisica di LaCantina.



3.4. *Persistent data management*

La gestione dei dati persistenti si articola su cinque entità principali:

- **Utente** (id, email, password hashata, nome, cognome, ruolo, ecc.)
- **Prodotto** (id, nome, descrizione, prezzo, quantità disponibile, provenienza, azienda fornitrice)
- **AziendaFornitrice** (id, nome, indirizzo, dati di contatto)
- **Ordine** (id, utente, data, importo, stato)
- **RigaOrdine** (id, id_ordine, id_prodotto, quantità, prezzoUnitario)

Dal punto di vista tecnologico, la persistenza è affidata al DBMS relazionale **MySQL**. L'interazione con la base di dati avviene tramite **JDBC** (utilizzando *PreparedStatement* per sicurezza ed efficienza), implementando il pattern **DAO (Data Access Object)** per ciascuna entità al fine di disaccoppiare la logica di business dalle operazioni di accesso ai dati.

3.5. *Access Control and security*

Attori e ruoli

- Utente non autenticato
- Può: visualizzare il catalogo (se previsto), registrarsi, autenticarsi.
- Non può: accedere a carrello, ordini personali, aree admin.
- Utente autenticato (ruolo: USER)
- Può: usare il carrello, fare ordini, consultare i propri ordini.
- Amministratore (ruolo: ADMIN)
- Può: modificare stock, prezzi, aggiungere/eliminare prodotti, modificare stato ordini.

Meccanismi tecnici

- Dopo il login riuscito, nella HttpSession viene registrato l'oggetto Utente e il suo ruolo.
- Le pagine e i controller che richiedono un certo ruolo:
- verificano la presenza dell'utente in sessione;
- verificano il ruolo corretto (USER o ADMIN);
- in caso di mancanza di permessi, reindirizzano a pagina di errore o login.

Protezione dati

- Password memorizzate solo in forma hashata.
- Parametri per query SQL passati tramite PreparedStatement.
- Controllo e validazione degli input di form (formato email, numeri, valori di prezzo, ecc.).

3.6. *Global Software control*

Il flusso di controllo globale del sistema segue un modello event-driven, tipico delle architetture web, dove ogni richiesta HTTP corrisponde a un evento scatenato da un'azione utente. Le Servlet agiscono come Controller: intercettano tali richieste, interpretano i parametri, invocano i servizi di business necessari e determinano la vista da restituire. La gestione della concorrenza è delegata interamente al container Apache Tomcat, che serve ogni richiesta su un thread separato, sollevando l'applicazione dalla necessità di creare o gestire thread manualmente.

3.7. *Boundary Contitions*

- **Avvio del sistema (Initialization):** All'avvio del sistema, il container Apache Tomcat provvede al caricamento e al deploy del pacchetto “LaCantina.war”. Durante questa fase critica vengono istanziate le risorse necessarie al funzionamento dell'applicazione, inclusa l'inizializzazione del pool di connessioni verso il database e il caricamento in memoria dei parametri di configurazione globali, quali la stringa di connessione JDBC e l'URL dell'endpoint per il gateway di pagamento.
- **Spegnimento del sistema (Termination):** Nella fase di spegnimento, il container arresta l'esecuzione dell'applicazione web avviando le procedure di rilascio delle risorse. Questo processo assicura la chiusura ordinata delle connessioni al database e l'interruzione di eventuali thread o processi schedulati, provvedendo contestualmente all'invalidazione immediata di tutte le sessioni utente ancora attive per garantire la sicurezza e la coerenza dei dati.
- **Gestione errori e failure (Failure Handling):** Il sistema è progettato per intercettare errori applicativi o eccezioni impreviste, registrando i dettagli tecnici nei log del server per finalità diagnostiche. Per preservare l'integrità dei dati, qualora il guasto si verifichi durante un'operazione di persistenza, viene eseguito automaticamente il rollback della transazione attiva; parallelamente, all'utente viene mostrato un messaggio di errore generico o una pagina di cortesia, nascondendo i dettagli interni dell'eccezione per motivi di sicurezza e usabilità.

4. **Subsystem services**

- **Servizi del sottosistema UserManagement:** Il sottosistema espone le funzionalità essenziali per la gestione del ciclo di vita dell'identità utente. Il servizio login(email, password) si occupa dell'autenticazione verificando le credenziali, determinando il ruolo dell'utente (standard o amministratore) e inizializzando la sessione di lavoro. Parallelamente, logout() gestisce la terminazione sicura della sessione attiva. Per i nuovi utenti, register(datiUtente) permette la creazione di un account, validando i dati in ingresso e garantendo l'univocità dell'email nel sistema.
- **Servizi del sottosistema CatalogManagement:** Questo sottosistema fornisce l'accesso in lettura alle informazioni sui prodotti. Il metodo getAllProducts() restituisce l'elenco completo degli articoli disponibili per popolare le viste del catalogo, mentre getProductDetails(idProdotto) permette di recuperare la scheda dettagliata di un singolo articolo, includendo informazioni specifiche quali descrizione estesa, prezzo, provenienza, azienda fornitrice e giacenza attuale.
- **Servizi del sottosistema Cart:** Il modulo gestisce lo stato transitorio del carrello acquisti memorizzato in sessione. Tramite addProduct(idProdotto, quantità) è possibile aggiungere nuovi articoli o incrementarne la quantità, previa verifica della disponibilità di stock. Le modifiche sono gestite da updateQuantity(idProdotto, nuovaQuantità), che aggiorna le quantità sempre nel rispetto dei limiti di magazzino, e da removeProduct(idProdotto), che elimina un articolo dalla lista. Infine, emptyCart() provvede allo svuotamento completo del carrello, operazione tipicamente invocata dopo la conferma di un ordine.
- **Servizi del sottosistema OrderManagement:** Il sottosistema centralizza la logica di business relativa agli ordini. La funzione principale createOrderFromCart(carrello, datiSpedizione, datiPagamento) converte il contenuto del carrello in un ordine persistente, registrando le righe, aggiornando le giacenze di magazzino e verificando l'esito del pagamento. Per la consultazione, getOrdersByUser(idUtente) restituisce lo storico

cronologico degli acquisti di un utente, mentre `updateOrderStatus(idOrdine, nuovoStato)` fornisce all'amministratore lo strumento per tracciare l'avanzamento dell'ordine (es. passaggio a "spedito").

- **Servizi del sottosistema Administration:** Dedicato alle operazioni privilegiate di backend, questo sottosistema consente la manipolazione completa del catalogo. L'inserimento di nuove referenze avviene tramite `addProduct(datiProdotto)`, mentre la gestione degli articoli esistenti è affidata a `updateStock(idProdotto, nuovaQuantità)` per l'aggiornamento dell'inventario e `updatePrice(idProdotto, nuovoPrezzo)` per le variazioni di listino. È inoltre prevista la funzionalità `deleteProduct(idProdotto)` per la rimozione di articoli dal catalogo, subordinata al rispetto dei vincoli di integrità del business.